Pascal VOC object detection - technical report

Author: Faik Bairamov **Date:** 29 October 2026

1. Problem and delivered solution

Detect and localize 20 everyday object categories in a new photograph. The project covers data validation, transfer learning, controlled experiments, evaluation, optimization, and a working deployment. FastAPI serves an image-upload interface, confidence slider, labelled boxes, an image download and structured JSON. A CLI and a CPU-only Docker image provide additional ways to run the same model.

The final selected model scores **74.94% development mAP@0.5** on 1,144 images and **74.17% local held-out validation mAP@0.5** on 5,823 images. Development improved by **2.02 percentage points** over the starting epoch-3 model (72.92%). Both final scores were freshly recomputed from the exact exported weights used by the demo.

2. Data, preparation and architecture

Pascal VOC 2012 offers multiple annotated objects per photograph and a manageable 20-class benchmark for CPU/GPU transfer learning. The official train list contains 5,717 images. A scikit-learn image-level 80/20 split, seed 42, yields 4,573 fitting and 1,144 development images. Sorted manifests are committed. Official val contains 5,823 images; test-image archives have no usable local labels and were not used.

The audit decoded all 11,540 train/val images, checked XML dimensions, known labels, box bounds, unique/disjoint split IDs, and the fit/dev partition. Zero errors were found. This establishes ID isolation, not absence of near-duplicates or overlap with pretraining data. Class imbalance is retained and documented in the data CSVs and distribution figure.

Convert one-based inclusive VOC boxes to zero-based exclusive-end coordinates by subtracting one from xmin/ymin and retaining xmax/ymax. RGB images become tensors in [0,1]; TorchVision performs normalization and resizing. Difficult objects remain in training and are treated specially by evaluation. Horizontal flips update both pixels and x coordinates; tests check geometry and prevent augmentation in dev/val. The selected checkpoint's flip probability is 0.5, and its training minimum-side settings are [384]. The original baseline used resizing/normalization only.

The detector is TorchVision Faster R-CNN MobileNetV3-Large 320 FPN, COCO pretrained, with a new 21-class predictor (20 objects plus background). MobileNet/FPN features feed a region proposal network and ROI classification/box-regression heads. The architecture was chosen for an existing supported CPU-friendly transfer-learning pipeline. The pretrained variant name includes 320; the final inference settings are explicitly **384/768 pixels** and **300 RPN proposals**. Model class order and these settings are stored with the weights. COCO already contains related categories, so this is transfer learning rather than learning object concepts from scratch.

3. Experiments and decisions

Training uses SGD, batch size 2, momentum 0.9 and weight decay 0.0005. Initial learning rate is 0.001; the successful continuation uses 0.0001. MLflow stores parameters, losses, dev AP, split artifacts and checkpoints. The following table contains actual tracked results; each training experiment is one seed.

Experiment	Device	Eval min / RPN	Train loss	Dev mAP@0.5
Baseline epoch 1	cpu	320 / 150	1.0607	69.84%
Baseline epoch 2	cpu	320 / 150	0.8142	70.66%
Epoch 3 control; reset shuffle	cpu	320 / 150	0.7139	69.99%
Epoch 3 lower LR; reset shuffle	cpu	320 / 150	0.6203	72.92%

Experiment	Device	Eval min / RPN	Train loss	Dev mAP@0.5
Epoch 2 weights; 640 resolution	cpu	640 / 150	-	52.03%
mps-epoch4-control	mps	320 / 150	0.5876	73.19%
mps-epoch4-horizontal-flip	mps	320 / 150	0.6383	73.15%
mps-multiscale-continuation	mps	320 / 150	0.5583	73.08%
mps-multiscale-continuation	mps	320 / 150	0.4988	73.17%
mps-epoch7-fixed384	mps	320 / 150	0.4936	72.68%

The historical epoch-3 control and lower-LR run start from the same epoch-2 weights and optimizer with reset seed-42 shuffle. Early checkpoints lack RNG state; this is not an exact uninterrupted third epoch. Lower LR improved AP despite the control's lower loss failing to improve AP. Training loss alone does not decide model quality. New continuations preserve Torch/loader RNG and MPS RNG when used. CPU/GPU numeric and random differences prevent exact cross-device reproduction.

The computer's Apple M5 supports PyTorch MPS. A temporary-model device benchmark measured mean warmed-up steps of 0.3911 s on CPU and 0.1615 s on MPS over five fixed-minibatch timings, with a seven-second first GPU step. These measurements motivated actual GPU fine-tuning; they are not a full-epoch speedup guarantee. All training candidates are scored using CPU inference on the fixed development split for parity with the supported deployment.

A free Colab Tesla T4 was allocated and a CUDA mixed-precision attempt was run, but it slowed with system RAM near the free runtime limit and did not finish a scored checkpoint before it was stopped for the deadline. No cloud accuracy or speedup is claimed; it contributed no selected weights. The attempt is documented in `reports/cloud_attempt.md`. No paid compute or subscription was used.

Inference settings were also compared on development:

Experiment	Min/max side	RPN proposals	Dev mAP@0.5
epoch4 control 384	384/768	300	74.21%
epoch4 control 384 proposals150	384/768	150	73.90%
epoch4 flip 384	384/768	300	74.55%
epoch4 flip 448	448/896	300	73.84%
epoch5 multiscale 384	384/768	300	74.53%
epoch6 multiscale 384	384/768	300	74.86%
epoch7 fixed384	384/768	300	74.94%
proposals75	320/640	75	72.31%
resolution320 proposals300	320/640	300	72.90%
resolution384 proposals300	384/768	300	73.77%

Experiments include unsuccessful changes as well as improvements. Historical 640/1280 inference on epoch-2 weights scored 52.03%, showing that larger inputs are not automatically better without suitable training. Model weights and final inference settings were chosen using development only. The selected run is `63411f882d0d45898a5877fa40e2c0c9`, epoch 7; `reports/model_selection.md` records the final decision before its validation run.

4. Evaluation and errors

The custom metric is VOC all-points interpolated AP with strict IoU > 0.5, confidence-ranked detections, one match per non-difficult object, duplicates counted as false positives, and matches to difficult annotations ignored. The arithmetic mean over 20 categories is mAP@0.5. Synthetic tests cover matching, duplicates, difficult objects, empty detections, false-first rankings and the IoU boundary. Full numerical parity with the official VOC devkit has not been independently established. This is not COCO mAP@[.5:.95] or an official test-server result.

AP uses all returned detector predictions, including its internal score filtering, NMS and per-image detection cap; it does not apply the demo's 0.5 display filter. The validation result is **74.17%**. Stronger validation categories: cat (89.21%), train (85.55%), dog (85.33%). Weaker categories: pottedplant (43.33%), bottle (48.79%), boat (55.06%). Full per-class CSVs and PR plots accompany the report. Checkpoint SHA-256: `f4c5318a5e574007e9473894f277333a302252d7e0f28514499543cbce70cb5c`.

Development diagnostics at confidence 0.5 match 1908 of 2732 non-difficult objects (69.84% recall), miss 824, and ignore 73 difficult matches. Retained false positives include 24 duplicates, 129 class confusions, 297 localization errors and 138 background detections. The diagnostic taxonomy uses best IoU > 0.5 for class confusion, > 0.1 for localization, otherwise background, after correct-class matching. This heuristic analysis is distinct from AP. For original-image box area below 32 x 32 pixels, 145 of 160 objects are missed (9.38% recall). This is not COCO AP-small. Saved bottle/pottedplant figures show annotations and predictions for direct inspection.

At the same 0.5 display threshold the starting model matched 1,812 objects and had 429 retained false positives. The final model matches 96 more objects but has 159 more false positives. Improved ranking AP does not guarantee improved precision at one fixed score threshold; the interface exposes that tradeoff through its slider.

The earlier epoch-3 model was already evaluated once on val (71.84%) before the additional requested improvements. New training and selection used development only, followed by this final val evaluation. Thus val is a held-out training split, but it is not an untouched independent test after all project iterations. No pretraining-overlap audit or statistical-significance claim is made.

5. Optimization, deployment and verification

The final full training checkpoint is 152,329,920 bytes; the exported FP32 inference checkpoint is 76,406,651 bytes, a 49.84% file-size reduction from removing optimizer/training state. This does not shrink in-memory FP32 parameters or by itself accelerate inference. Model/split hashes and complete configuration are saved with the scores.

Separate optimization experiments on the epoch-3 baseline are preserved in `reports/optimization.md`: 75 proposals reduced AP without improving mean latency; dynamic INT8 linear heads reduced comparable state-dict storage from 76.42 MB to 34.42 MB but increased mean forward latency from 60.68 to 64.54 ms. Convolutions remain FP32. The 0.037-point AP change is not evidence of a robust accuracy gain. Legacy torch.ao quantization emits deprecation warnings and is isolated from the supported FP32 serving path. ONNX, pruning and full-network INT8 are future work. Those baseline measurements must not be attributed to the new selected weights.

Final selected-model CPU profiling uses 80 fixed development images, 3 repeats, five warmups and batch size one: mean **69.45 ms**, median **68.01 ms**, p95 **78.46 ms** with 4 threads. It times model forward on decoded tensors, excluding image decoding, drawing, base64 encoding, transport and concurrent service load. Single-machine results are not a production SLO or a controlled cross-device benchmark.

FastAPI validates file bytes/pixels and confidence values, returns a typed response, and serializes access to one loaded CPU model. The health endpoint exposes loaded model provenance. Docker uses CPU-only Torch wheels, an unprivileged user and a health check. The final CLI, API and Linux ARM64 container are checked with real image bytes; contract tests also exercise invalid input. Browser upload behavior was inspected. Exact check evidence is under `reports/verification/`. No dataset or network access is required for inference after setup. Other platforms and production concurrency/authentication are not claimed as verified.

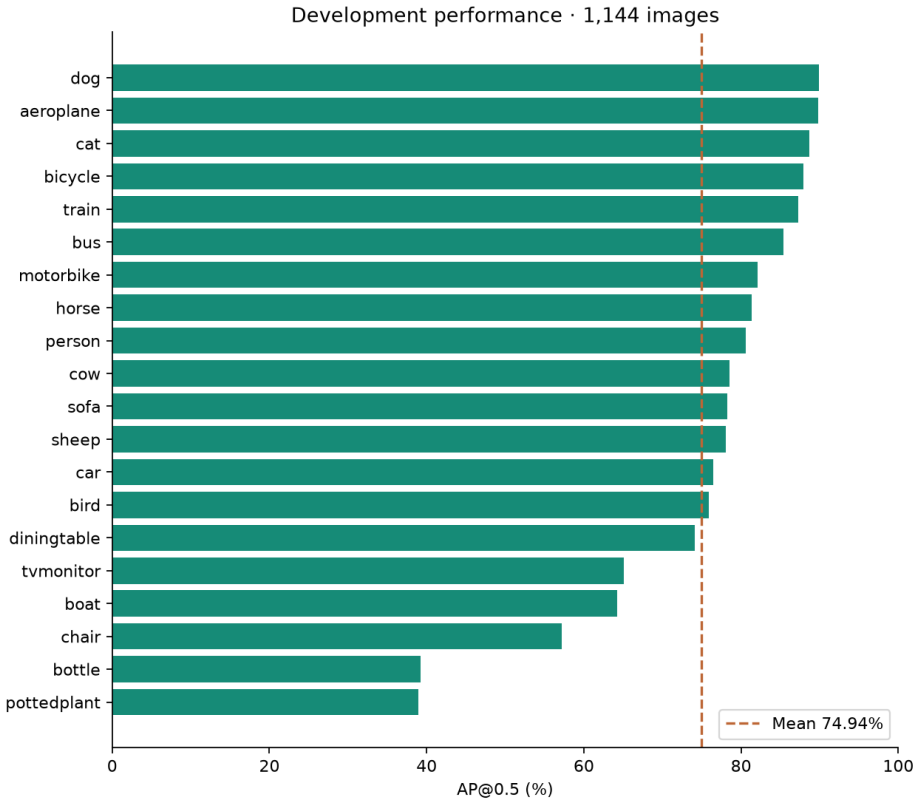
6. Limitations, next work and AI use

Strengths are a runnable end-to-end system, real GPU experiments, explicit rejected experiments, preserved checkpoints, traceable evaluation, error diagnostics and a working deployment. Limitations include one seed per training experiment, benchmark and class imbalance, weak small-object performance, custom metric parity, unknown pretraining overlap, and limited cross-platform validation. High confidence is not a calibrated probability of correctness on arbitrary deployment images.

With more time, compare multiple seeds and a YOLO baseline under identical data and metrics; investigate scale-aware sampling, calibration and out-of-domain data; verify official-devkit parity; evaluate supported TorchAO/ONNX optimization on the deployment CPU; and add concurrent load tests. Keep future tuning on development or establish a new independent test protocol.

Codex helped explain/refactor the workflow and drafted inference/API/deployment, analysis code, tests and documentation, and executed the recorded checks under the author's direction. This is not independent human review or proof of complete correctness. `reports/ai_use.md` discloses contributions; the author must understand and defend the submitted code. The README supplies environment, data, training, continuation, evaluation and run commands; the ZIP supplies the exact weights.

Performance by category



Development AP by category. Additional precision-recall curves and error images accompany the report under reports/figures/.